TFG · SONDA PASIVA DE DETECCIÓN PQC · TAREA 3

libpcap o Scapy: decisión y validación cruzada

Scapy como generador de trazas y como segunda implementación para validar la sonda en C

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **04/11/2026** | Tutoría del miércoles 04/11/2026

1. Conclusión principal

> **RESULTADO**
>
> **Se mantiene la decisión de la tutoría 1: la sonda va en C sobre libpcap, y Scapy se usa para validar.** Ahora con datos: sobre 3000 conexiones, la sonda y una implementación independiente en Scapy dan **exactamente el mismo resultado en las 3000**, pero la sonda tarda **0,116 s** y Scapy **19,7 s**, unas **170 veces más**. Además, el disector TLS de Scapy no interpreta el HelloRetryRequest, que es justo el mensaje que distingue a un servidor que fuerza PQ.

2. Comparativa

Criterio	C + libpcap (sonda)	Python + Scapy
Velocidad medida	**≈ 200 000 paquetes/s** (23 000 en 0,116 s)	≈ 1 200 paquetes/s (23 000 en 19,7 s)
Captura en vivo	Viable en enlaces reales	Pierde paquetes con tráfico moderado
TLS 1.3 / HRR	Parser propio: lee ClientHello, HRR y ServerHello	Disector completo, pero **deja el HRR sin interpretar** (bytes en crudo)
Control de memoria	Total; validado con AddressSanitizer	Lo gestiona Python
Requisito del TFG	Cumple: sonda en C sobre libpcap	No cumple
Fabricar paquetes	Laborioso	**Muy cómodo**: es su punto fuerte
Uso en el TFG	**La sonda**	**Generar trazas de prueba y validar** la sonda

3. Las dos herramientas (en `herramientas/`)

`gen_trazas.py`: Scapy fabricando tráfico

Construye los mensajes TLS byte a byte según la RFC 8446, con las longitudes reales de las claves (X25519MLKEM768: 1216 B en el cliente y 1120 B en el servidor), y los mete en conexiones TCP completas (SYN, datos, FIN). Usa semilla fija, de modo que cada ejecución produce exactamente las mismas trazas.

Traza	Contenido
`casos_clasificacion.pcap`	Una conexión por cada situación de clasificación (6 casos de la tarea 2)
`reensamblado_desordenado.pcap`	ClientHello en 3 segmentos: desordenado y con una retransmisión
`rendimiento.pcap`	3000 conexiones (los 6 casos × 500), 23 000 paquetes; para medir velocidad

Se ha corregido un defecto de la versión anterior: Scapy asigna la marca de tiempo al *crear* cada paquete, y al desordenarlos después la columna Time de Wireshark iba hacia atrás. Ahora los tiempos se asignan en el orden final, como en una captura real (comprobado en Wireshark).

`validar_scapy.py`: Scapy leyendo tráfico

Lee la traza con el reensamblado TCP (`TCPSession`) y el disector TLS de Scapy, que no comparten código con la sonda, y extrae lo mismo: grupos que el cliente sabe, claves que manda, HRR y grupo elegido. Con `--sonda` ejecuta también la sonda y compara conexión a conexión.

```
python3 validar_scapy.py ../trazas/rendimiento.pcap --sonda ../sonda/sonda
```

Hallazgo: el disector TLS de Scapy no decodifica el HelloRetryRequest sin estado de sesión y lo deja como Raw. El validador lo lee a mano: comprueba el *random* fijo del HRR y recorre sus extensiones hasta el `key_share`.

4. Resultados

Traza	Conexiones iguales	Sonda	Scapy	Relación
`casos_clasificacion.pcap`	6 de 6	0,002 s	0,04 s	15×
`reensamblado_desordenado.pcap`	1 de 1	0,002 s	0,01 s	4×
`rendimiento.pcap`	**3000 de 3000**	**0,116 s**	**19,70 s**	**170×**

En las trazas pequeñas el tiempo es sobre todo el arranque del programa; la cifra representativa es la de 3000 conexiones.

Triple comprobación sobre `rendimiento.pcap`

Dato	Wireshark	Sonda
Paquetes	23 000	23 000
Conexiones	3000 (Estadísticas → Conversaciones → TCP)	3000 conexiones TLS
HelloRetryRequest	500 (filtro `tls.handshake.extensions_key_share_selected_group`)	500 servidores «fuerza PQ»

Clasificación de la sonda: conexiones segura 1000 / insegura 2000; clientes inseguro 500 / potencialmente inseguro 2500; servidores inseguro 1000 / indeterminado 1000 / soporta PQ 500 / fuerza PQ 500. Cada cifra corresponde a los casos de la traza multiplicados por 500, y los registros TLS (8000) a 2 por conexión más 4 extra en cada conexión con HRR.

> ★ PARA COMENTAR EN LA TUTORÍA
>
> La propuesta de Jorge de mantener Scapy para validación se ha llevado a la práctica: **tres implementaciones independientes** (la sonda en C, Scapy y Wireshark) coinciden sobre las mismas trazas. Scapy es **unas 170 veces más lento** que la sonda y no interpreta el HelloRetryRequest, lo que confirma con datos la elección de C + libpcap para la sonda.

5. Notas

- `rendimiento.pcap` (7,5 MB) no se sube al repositorio: se regenera con `gen_trazas.py` en unos segundos y siempre sale igual.
- Las trazas son sintéticas. Falta validar con tráfico real con PQ (Chrome capturado por la interfaz Wi-Fi de Windows con Npcap).
- Medidas en el portátil del autor (Ubuntu en WSL2); la relación de tiempos es lo relevante, no los valores absolutos.